

Grupo 3 - Módulo Robô de Lances

Integrantes: Eliéder Rosinski Marques, Giovanni Leote Ferreira, Laura Beatriz Kappel Waechter, Pedro Henrique Glesse Baumhardt e Vinícius Rehbein Kobs

ADR 1 - Padrão Arquitetural e Stack Tecnológica

DATA: 10/05/2026

STATUS: aceito

CONTEXTO

O projeto consiste no desenvolvimento de um Módulo Robô de Lances, que exige alta confiabilidade, facilidade de manutenção e capacidade de resposta rápida para processamento de lances. O problema central é definir uma estrutura de código que permita separar claramente a lógica de negócio (o robô) da interface de usuário e do acesso a dados, garantindo que a equipe possa trabalhar de forma paralela e organizada.

ALTERNATIVAS CONSIDERADAS

Opção 1 - Arquitetura Monolítica Simples: Todo o código (Front e Back) em um único repositório e processo, utilizando renderização no servidor (Blade). Facilitaria o deploy inicial, mas dificultaria na escalabilidade.

Opção 2 - Arquitetura Desacoplada (API REST MVSC + SPA): Separação total entre Backend (Laravel) e Frontend (Vue3). O backend utiliza o padrão MVSC (Model-View-Service-Controller) para isolar a lógica complexa de lances em camadas de serviço.

DECISÃO

Opção 2. Escolhemos a stack **PHP/Laravel (Backend)** e **Vue3 (Frontend)**. A adoção do padrão **MVSC** no backend é justificada pela necessidade de extrair a lógica pesada do Robô de Lances para *Services*, permitindo maior manutenibilidade. A escolha também é fundamentada no conhecimento técnico prévio de alguns integrantes do grupo com essa stack e padrão, o que reduz a curva de aprendizado e mitiga riscos de atraso. O uso do Vue3 com *Composition API* no frontend garante uma interface reativa para o acompanhamento dos lances em tempo real.

CONSEQUÊNCIAS

Positivas: Clara separação de responsabilidades; maior escalabilidade (o backend pode ser escalado independentemente do frontend); código mais limpo e organizado através de camadas de serviço;

Negativas: Maior complexidade inicial na configuração do ambiente (CORS, autenticação JWT/Sanctum); necessidade de gerenciar dois ambientes de build/deploy distintos.

Mitigação: Uso de **Docker** para padronizar o ambiente de desenvolvimento entre todos os integrantes do grupo e automatizar o deploy de ambos os serviços.

ATRIBUTOS DE QUALIDADE

Ao optar por AWS para arquitetura conseguimos manter a baixa latência do robô e SMTP possibilita a entrega rápida de mensagens. Uma boa codificação bem estruturada também é capaz de proporcionar a permanência em diversas licitações diferentes.